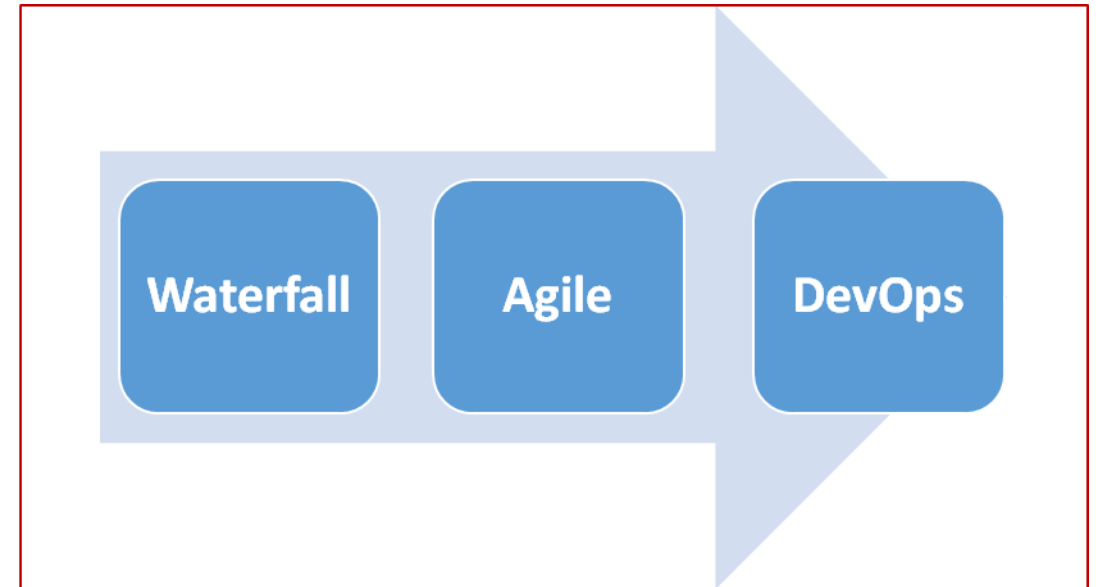


Chapter 1

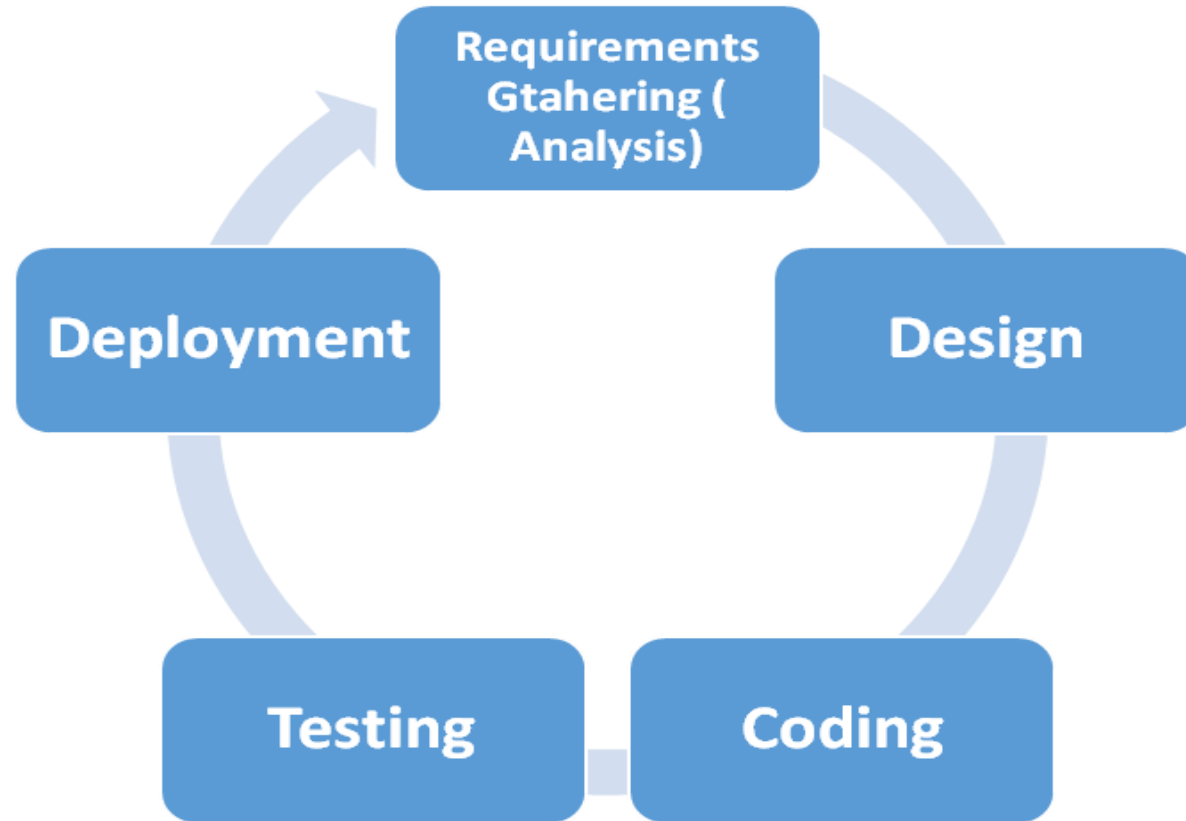
Introduction to DevOps

Evolution of software development

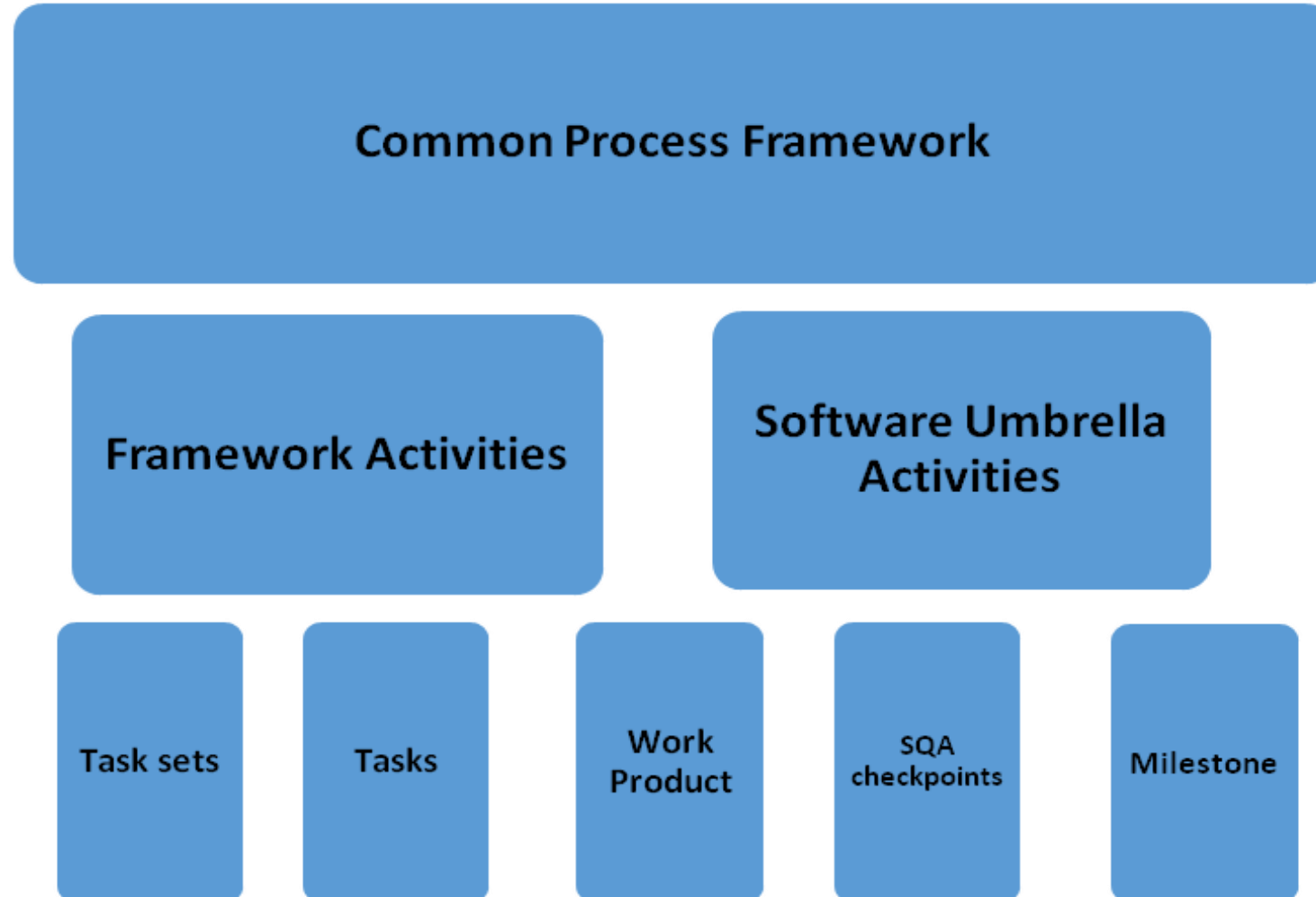
- **Traditional approach**
- **Agile development**
- **Agility in operations ?**
- **Continuous changes in requirements**
- **Developments & operations both need to be agile**



Traditional Software development Approach:



Software Process Framework:

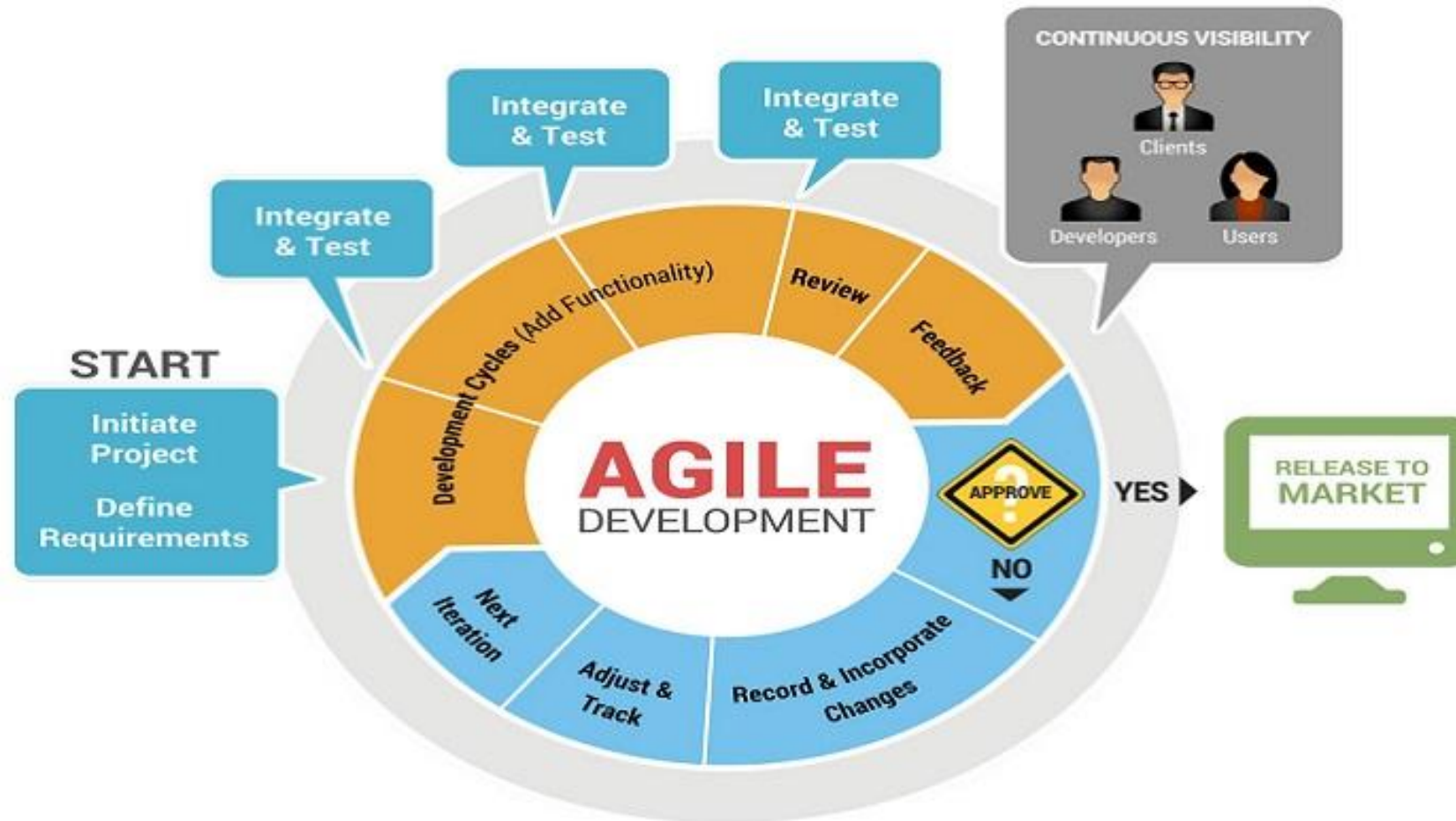


Pros & Cons of Traditional software Development Approach

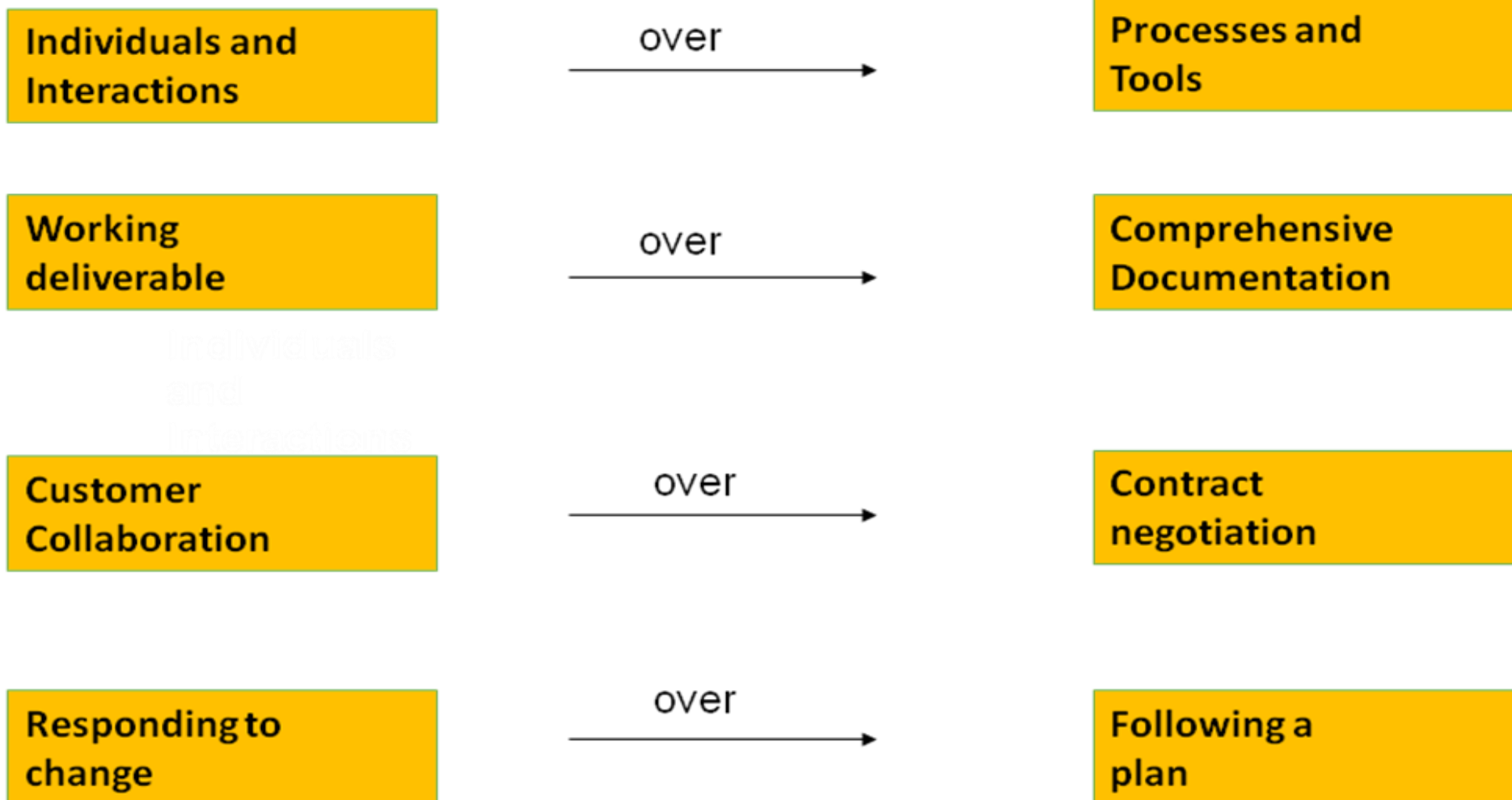
- **Pros: Stable requirements**
- **Cons: Never backward approach**

Example: Government software projects often follow the waterfall model due to strict requirements.

Agile Methodology for Software Development



Agile Manifesto



Agile Methodology for Software Development contd..

- Example: Scrum teams at Spotify use agile practices for rapid delivery and innovation.
- **Microsoft's Windows Team** : Microsoft historically followed the Waterfall model but shifted to Agile (Scrum) during the development of Windows 10.
- **They adopted shorter sprint cycles, continuous feedback, and released frequent updates rather than waiting years for a major launch. This allowed quicker bug fixes and feature enhancements.**

INTRODUCTION TO DEVOPS

- Is a combination of two words 'development' and 'operations.'
- never-ending process of continual improvement.”
- **culture** that we can adopt for continuous improvement
- help to bring your **Developer Team** and **Operations Team** on the same page,
- work together with better **communication** and **collaboration**.

INTRODUCTION TO DEVOPS *CONTD..*



DevOps is a product development culture which promotes collaboration between Development and Operations Team to deploy code faster through automated & repeatable development.

Example: Netflix uses DevOps to rapidly deploy code changes to production multiple times a day.

History & Origins of the DevOps

- concept of computer originated from “Turing machine” in 1936
- Fortran programming language in 1957
- late 1970s and 1980s, when CRT monitors
 - Developers write code in development, compile it, and test it on the same computer.
 - Deploying code consisted of compiling, copying to dozens of flop disks, and repeating.
 - Companies started advertising for programming positions to write source code or programs.
- After the origin of Internet new engineering jobs were created for “production environment”.
- conflict were created between developers and the operations engineers.
- DevOps evolved from the need for better collaboration between development and operations teams. It gained popularity around 2009 after the 'DevOps Days' conference in Belgium.
- Example: Flickr's deployment of 10+ times a day inspired the early DevOps movement.

DevOps Objectives

Faster Delivery



Increase the frequency and pace of releases so you can innovate and improve your product faster.

Continuous integration and continuous delivery are practices that automate the software release process, from build to deploy.

Improved Collaboration



Build more effective teams under a DevOps cultural model, which emphasizes values such as ownership and accountability. Developers and operations teams collaborate closely, share many responsibilities, and combine their workflows. This reduces inefficiencies and saves time (e.g. reduced handover periods between developers and operations, writing code that takes into account the environment in which it is run).

DevOps Objective *Contd..*

Better Quality & Reliability



Ensure the quality of application updates and infrastructure changes so you can reliably deliver at a more rapid pace while maintaining a positive experience for end users.

Use practices like continuous integration and continuous delivery to test that each change is functional and safe.

Monitoring and logging practices help you stay informed of performance in real-time.

Security



Move quickly while retaining control and preserving compliance. You can adopt a DevOps model without sacrificing security by using automated compliance policies, fine-grained controls, and configuration management techniques. For example, using infrastructure as code and policy as code, you can define and then track compliance at scale.

DevOps Objective *Contd..*

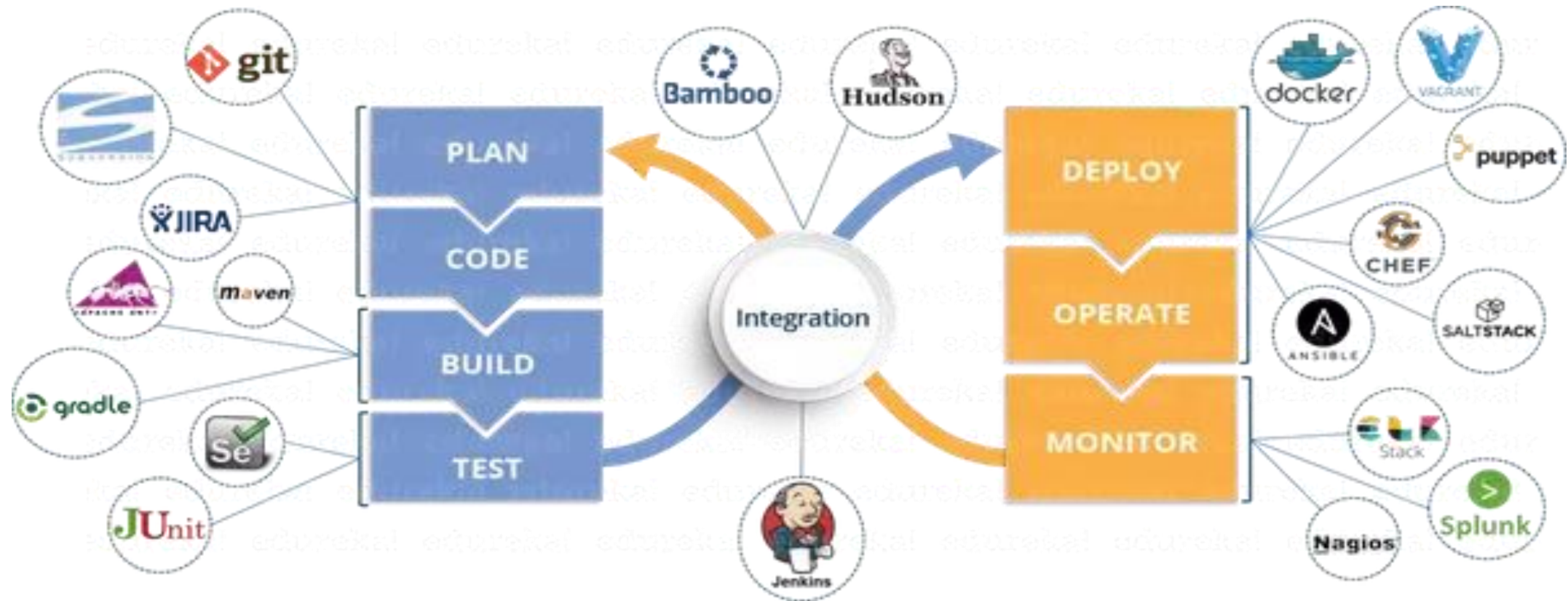
Example: Amazon achieves faster time to market by aligning business goals with IT operations.

This allows for:

- **Rapid innovation**
- **Frequent, safe deployments**
- **High system reliability**
- **Agile response to market demands**

That's how Amazon stays ahead in time-to-market across its global business lines.

DevOps Delivery Pipeline



Includes stages like build, test, release, deploy, operate, and monitor.

Example: Facebook's delivery pipeline allows thousands of code commits and tests every day.

1. Plan (Collaborate) – Where the team decides what to develop next.

Example: Product managers, developers, and operations collaborate in Jira to plan the next sprint.

2. Develop (Code) – Where code is implemented by developers.

Example: A developer writes a new “login” feature in their IDE and pushes it to Git.

3. Build – Where code is compiled, packaged, and artefacts are created.

Example: Jenkins pulls from Git and compiles Java code into a .jar.

4. Test – Where automated tests validate functionality and find bugs early.

Example: Unit, API, and UI tests run in a pipeline to check for regressions.

5. Release (Deploy) – Where code is deployed to staging or production.

Example: Ansible or Docker Compose publishes the build to a staging server.

6. Operate – Where deployed application runs smoothly and is maintained.

Example: Kubernetes handles scaling and load distribution in production.

7. Monitor – Where performance and health are constantly tracked.

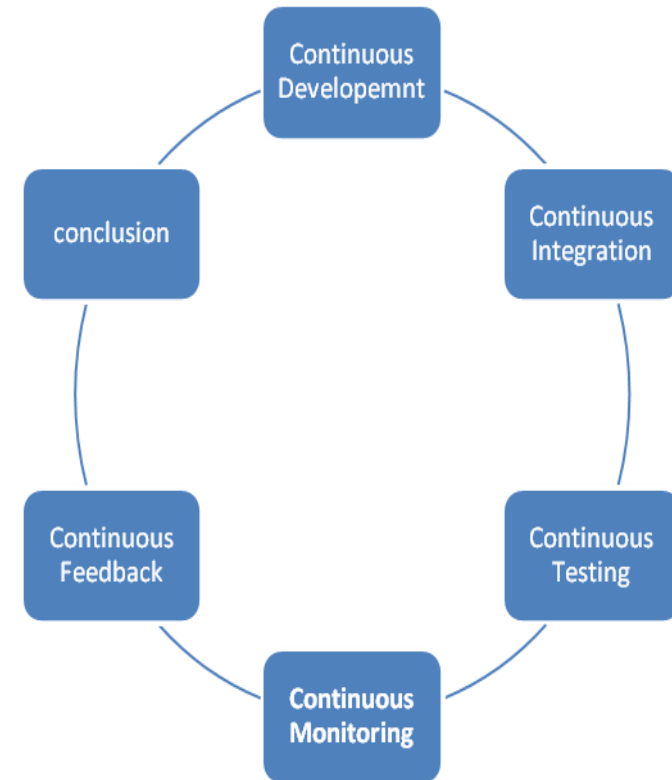
Example: Grafana and Prometheus alert you if CPU usage crosses 90%.

8. Feedback – Where stakeholders review and provide input for future cycles.

Example: User reviews and error reports are fed back into Jira for future sprints

Lifecycle

- **Continuous Development**
- **Continuous Testing**
- **Continuous Integration**
- **Continuous Deployment**
- **Continuous Monitoring**



***Lifecycle** contd..*

- **Continuous Development:** Writing and updating code frequently.

Example: Developers adding new features daily and pushing code to Git.

- **Continuous Integration:** Merging code into a shared repository and build it automatically.

Example: Git commits triggering a Jenkins build to combine all code.

- **Continuous Testing:** Running automated tests to validate code functionality.

Example: Unit and API tests execute after each build to find bugs.

- **Continuous Deployment:** Deploying code to production once it passes all checks.

Example: Docker and Ansible automatically release the application to a staging or production server.

- **Continuous Monitoring:** Continuously observing application health and performance.

Example: Grafana and Prometheus track CPU, memory, and error rates in real time.

Challenges DevOps solves

- **Deployments taking too long:** Automates delivery to make releases faster and more reliable.
- **Missing artefacts, scripts and other dependencies:** Ensures all components are bundled and available when needed.
- **Confusion about what to test:** Provides a clear, automated testing pipeline to validate code.
- **Slow test processes:** Speeds up testing through automation and parallel execution.
- **Unavailability of environments:** Allows for on-demand, consistent, and reproducible test and staging environments.
- **Fire fighting instead of innovation:** Frees up teams from reactive fixes to focus on developing new features.
- **High dependency on specific individuals:** Reduces knowledge silos by standardizing processes and sharing responsibilities.

Tools

- **Source code management:** [Git](#), [SVN](#) and [JIRA](#)
 - Maintaining code throughout development lifecycle, version control
- **building:** [Maven](#) & [Gradle](#)
 - Package code to executable files
- **Testing :** [Selenium](#), [TestNG](#), [JUnit](#)/ [NUnit](#)
- **Continous integration:** [Jenkin](#), [Bamboo](#) and [Hudson](#)
 - Test changes, test that change with other changes
- **Continuous deployment :** [puppet](#), [chef](#), [saltstack](#), [Ansible](#), [docker](#)
 - Code is tested & ready pushed to production or non development machine
- **Continous monitoring :** [NewRelic](#), [sensu](#) , [Nagios](#)
 - Monitors servers closely & check health of the system
 - Anything happens to servelets it monitors s proactively rather than reactively

Case study

- **Let's suppose that a company is developing an eCommerce platform. For this, they have hired 4 developers to build the application and two engineers to handle the operations such as maintaining the infrastructure, production environment and providing support to the end-client.**
- **The four developers working on the first module would gather all their week's work at the end of the week in order to merge their codes manually and then test what they've built. Once the module is complete, they will execute the codes on their local test server and test the flow again to see if everything works fine.**
- **It is only then that they can move to the next module or else they will have to fix the ongoing module first before moving ahead. There's no other way because if they fail to address these issues then they'd have to wait for the entire cycle to finish and can only make changes in the next version release.**

Case study

- Now, let's say that the Development team after spending 9 more months finishes their task of creating an eCommerce platform. This includes development as well as basic level application testing on their local system to check whether everything works fine including client creation, order placement, order and payment processes, etc.
- The development team hands over their code to the operation team who are responsible for releasing the application for actual commercial use. It is here that the actual roller coaster ride begins.
- 'Roller Coaster ride' is a fancy phrase in the English language and the funny part is that its meaning isn't as good as it sounds because the word is everything but fun. Why did I use it here? It's because the perfectly tested working eCommerce application is in fact not so perfect. Upon testing with actual data, the operations team finds out that the application that took nine months to finish is unable to create clients properly, the files are missing, validation issues are popping out of nowhere, placed order payments are not being processed, etc.

Case study

- **The operations team is frustrated and angry because this is what they get after waiting for nine months. The development team, on the other hand, is now offended because as per their testing everything was working perfectly during controlled testing conditions and the royal mess now must be because of the operation's poor execution.**
- **What happens now? They'll have to wait for months again to undo all the mistakes and deliver those changes in the next release.**
- **All of these could have been avoided had the development and operations teams decided to work together right from the conception and execution to its delivery and support. Instead, they worked in silos with an imaginary wall between them.**
- **The problem is that the operations team was not involved when the code was being written to build the eCommerce application and the development team was completely out of the picture when the eCommerce application went into production and was taking orders and processing it, the result is an eCommerce application that isn't ready for the market and a dev team that still doesn't quite know how to resolve things.**

Case study

- **As you can imagine, a lot of time will go by before the application is finally ready for the production server. There will be a number of iterations as the development team will have to make changes and the operations team sends the application to the production. This same issue will keep on happening not only during the product development but also when the application needs changes, maintenance and upgrades. As a result, even small startups end up becoming slow and inefficient and there's a good chance that they might lose out to other companies that get superior products to market faster.**
- **This was the problem that companies started seeing, especially as technology became more advanced. Keeping their operations and development teams isolated from each other was resulting in slower delivery and less efficient products and services. Moreover, many processes in the company operations that could have been easily automated to increase efficiency were not being automated because the developers were simply unaware of them.**

Case study

- “That’s when the concept of **DevOps** came to the forefront and began being adopted widely.
- DevOps is nothing but a set of philosophies, practices, and tools that help an organization to deliver better products faster by facilitating the integration of the development and operations functions.
- This enables companies like the one in our example to serve their customers and markets in a better way and have a competitive edge.
- DevOps has become possible largely due to a combination of new operations tools and established agile engineering practices”

E Commerce Company Case Study

1. Title of the Case Study

Bridging the Gap Between Development and Operations: A Case for DevOps in eCommerce Application Development

2. Background/Context

A company sets out to build an eCommerce platform. It hires:

4 Developers for application development.

2 Operations Engineers for infrastructure, deployment, and production support.

The development team follows a **traditional approach** where code is written, tested locally, and then handed off to operations for deployment without any integration or collaboration between the two teams.

E Commerce Company Case Study contd..

3. Problem Statement

Despite nine months of development and testing, the application **fails in the production environment**:

Clients cannot be created.

Files are missing.

Validation errors appear.

Order payments fail to process.

The **Operations team** blames the **Development team**, and vice versa. Each team works in **silos**, resulting in:

Delays in release cycles.

High rework.

Frustration and finger-pointing.

Poor customer experience.

Competitive disadvantage.

E Commerce Company Case Study

4. Objectives of the Case Study

- To highlight the challenges of isolated development and operations.
- To understand the consequences of lacking collaboration.
- To introduce DevOps as a solution to bridge the gap.
- To showcase the impact of DevOps on product delivery and efficiency.

5. Key Challenges Identified

Challenge	Description
Manual Code Merge	Developers merge code weekly, causing delays and conflicts.
Isolated Testing	Testing happens only in local environments, not real-world conditions.
Siloed Teams	Dev and Ops teams don't collaborate, causing friction and inefficiency.
Late Feedback Loop	Bugs are found only after release to operations, resulting in rework.
Inefficiency	Long development cycles with poor automation and deployment delays.
Business Risk	Product is slow to market and unstable, risking business loss.

E Commerce Company Case Study contd..

6. Analysis

The project suffered due to:

Lack of Continuous Integration and Continuous Delivery (CI/CD)

No collaboration between Dev and Ops

Delayed testing with production data

No automation in deployment or environment management

This clearly demonstrates the **need for a DevOps culture** where collaboration, automation, and integration are key.

7. Solution: DevOps Approach

DevOps integrates development and operations through:

Shared responsibilities from design to deployment.

Automation tools (CI/CD, Infrastructure as Code).

Continuous feedback and improvement loops.

Agile practices adopted throughout the pipeline.

By using DevOps:

Developers get **immediate feedback** from real-world environments.

Operations can **automate deployments** and reduce manual errors.

Teams can **collaborate continuously**, increasing velocity and reliability.

E Commerce Company Case Study contd..

8. Outcomes/Benefits of DevOps Adoption

Benefit	Description
Faster Time to Market	Continuous integration and deployment reduce wait time.
Higher Product Quality	Continuous testing detects issues early.
Better Collaboration	Dev and Ops work together across the lifecycle.
Reduced Downtime	Automated monitoring and feedback loops reduce failures.
Competitive Advantage	Faster, reliable releases help win market share.

9. Conclusion

This case study illustrates how the **lack of collaboration and automation** can derail even a well-planned software project.

Adopting DevOps ensures smoother transitions, quicker delivery, and more robust applications. For modern engineering students and future professionals, understanding DevOps is **key to building scalable and market-ready software solutions**.

E Commerce Company Case Study contd..

10. Assignment

Students are required to **map DevOps tools and practices** that could solve each problem listed.

Discuss real-world companies that have benefited from DevOps adoption (e.g., Amazon, Netflix).

Facebook case Study

Situation

New Features: Facebook was introducing **Timeline and Music** — a huge change to its UI and functionality.

Mass Deployment: It was a **global rollout to over 500M users**, many of whom were highly active, adding enormous load on the website.

Impact: The rollout was causing **high website traffic and stability issues**, nearly “bringing the site to its knees.”

How DevOps Helped

Dark Launch: To avoid a large-scale failure, Facebook first deployed the new features to **a small subset of users** (dark launch).

This allowed engineers to **observe performance and bugs in real time** without affecting all 500M.

Continuous Monitoring: Using extensive monitoring and logging, Facebook kept a close watch on **load, errors, and user engagement**.

This made it easy to **quickly identify problems** and implement fixes.

Rapid Deployment: Once issues were resolved and stability improved, the rollout was gradually expanded to more and more users — a true **continuous delivery approach**.

Feedback Loop: Some users didn't respond well to certain features.

Because of their flexible architecture, Facebook was able to quickly disable or roll back those components. This made sure the overall service remained stable and reliable.

Facebook case Study contd..

Outcome

- **The rollout remained under control, thanks to dark launching, monitoring, and gradual deployment.**
- **User feedback was incorporated quickly, letting Facebook revert unwanted components while strengthening desirable ones.**
- **The result: a successful rollout to a massive user base without causing sustained outages or poor service.**

Key Takeaway

This case highlights how DevOps practices:

dark launching, continuous delivery, extensive monitoring, and fast rollback by enabling large-scale companies to innovate safely and respond quickly to problems in production.

How DevOps Works

- Development and operations teams are no longer “siloed.”
- Sometimes, these two teams are merged into a single team
- In some DevOps models, quality assurance and security teams may also become more tightly integrated with development and operations
- When security is the focus of everyone on a DevOps team, this is sometimes referred to as DevSecOps.

DevOps Practices

- The following are DevOps best practices:
- **Continuous Integration**
- **Continuous Delivery**
- **Microservices**
- **Infrastructure as Code**
- **Monitoring and Logging**
- **Communication and Collaboration**

Market Trend of DevOps

- **What is Driving the DevOps Market?**
- The demand for faster software delivery, improved collaboration between development and operations teams, and increased automation has made **DevOps a strategic priority** across industries.
- **Agile Development** and **Continuous Integration/Continuous Deployment (CI/CD)** are now the norm.
- Organizations want **reduced time-to-market, fewer failures, and rapid recovery** from issues.
- The push toward **digital transformation, cloud adoption, and AI integration** has further accelerated DevOps adoption.

Market Trend of DevOps *contd..*

Global Market Forecast

Metric	Detail
Market Size (2023)	~\$10.4 billion
Projected Market (2028)	~\$25–30 billion
CAGR	~20–25%
Key Sectors Adopting	BFSI, eCommerce, Healthcare, Telecom, EdTech, Manufacturing

Key Trends Shaping the DevOps Market

1. Rise of DevSecOps (Security Shift Left)

- Security is no longer a post-development task. It is integrated from the start.
- Example: **Netflix** uses open-source tools like **Security Monkey** and **Lemur** to automate security in its DevOps pipelines.

2. AI/ML in DevOps (AIOps)

- AI helps predict system failures, automate testing, and optimize resource usage.
- Example: **Facebook** uses **AI-driven monitoring** to auto-detect anomalies and fix deployment issues.

3. Kubernetes and Microservices

- Containerization has made DevOps easier, portable, and scalable.
- Example: **Spotify** uses **Docker and Kubernetes** to manage microservices, allowing teams to deploy multiple features per day.

4. DevOps on the Cloud

- Cloud platforms (AWS, Azure, GCP) offer built-in CI/CD, monitoring, and infrastructure provisioning.
- Example: **Amazon** uses its own AWS DevOps stack (CodePipeline, CodeBuild) to automate global deployments.

5. Platform Engineering

- Emergence of Internal Developer Platforms (IDPs) for self-service infrastructure.
- Companies are building custom platforms so Devs can deploy faster without waiting for Ops.

Industry-Wise DevOps Adoption

- **eCommerce:--** Continuous delivery of new features & bug fixes (e.g., **Flipkart, Amazon**)
- **Banking & Finance (BFSI):--** Real-time fraud detection, faster app releases (e.g., **JP Morgan, HDFC**)
- **Healthcare:--** Secure CI/CD for patient portals and telemedicine (e.g., **Mayo Clinic, Practo**)
- **Education/EdTech:--** Fast iteration of learning platforms (e.g., **Byju's, Coursera**)
- **Telecom:--** Automated network services & monitoring (e.g., **AT&T, Reliance Jio**)

Why is DevOps Trending Now More Than Ever?

- **Remote Work:--** Need for automation and collaboration tools
- **Cloud Growth:--** Easier access to scalable DevOps toolchains
- **Digital Transformation:--** Faster innovation cycles required
- **Startup Culture:--** MVPs and iterative releases demand agility
- **Customer Expectations:--** Continuous feature delivery and zero downtime

What is the DevOps Ecosystem?

- The **DevOps Ecosystem** is the complete set of **tools, practices, and stages** that enable continuous collaboration between development and operations teams for:
- **Faster software delivery**
- **Automation**
- **Improved quality**
- **Efficient monitoring and feedback**

Real-World DevOps Ecosystem Example: Netflix

Netflix has built one of the most successful DevOps ecosystems:

Function	Tool	Purpose
Source Control	Git	Code collaboration
CI/CD	Jenkins + Spinnaker	Fast delivery of microservices
Configuration Mgmt	Ansible	Automate setup of servers
Containerization	Docker + Titus (Netflix's own platform)	Containerized services
Monitoring	Atlas (in-house), Prometheus	System performance & alerts
Chaos Testing	Chaos Monkey	Test system resilience by intentionally breaking things

Why the DevOps Ecosystem is Important

- Enables **cross-functional collaboration**
- Automates **repetitive tasks**
- Supports **scalability and resilience**
- Helps organizations move from **weekly to daily deployments**
- **Reduces downtime and accelerates time-to-market**

DevOps Engineer Skills in the Market

Who is a DevOps Engineer?

- A **DevOps Engineer** is a technical professional who bridges the gap between software development (Dev) and IT operations (Ops). Their job is to enable **continuous integration, continuous delivery (CI/CD), automation, infrastructure management, and monitoring** to ensure faster and reliable software releases.

Key Skills Required for a DevOps Engineer

1. **Linux & Scripting Knowledge**
2. **Version Control Systems**
3. **CI/CD Pipeline Tools**
4. **Configuration Management**
5. **Containerization & Orchestration**

6. **Cloud Computing Platforms**
7. **Infrastructure as Code (IaC)**
8. **Monitoring & Logging**
9. **Security & DevSecOps**
10. **Soft Skills**

Example DevOps Engineer Job Description

- **Company:** Infosys

Role: DevOps Engineer

Skills Required:

- Git, Jenkins, Ansible
- Docker, Kubernetes
- AWS or Azure experience
- Python or Bash scripting
- CI/CD pipeline setup
- Monitoring (Prometheus, Grafana)

DevOps on the Cloud

- **DevOps on the Cloud means applying DevOps practices (automation, CI/CD, monitoring, infrastructure as code, etc.) using cloud computing platforms like:**
 - Amazon Web Services (AWS)
 - Microsoft Azure
 - Google Cloud Platform (GCP)
 - IBM Cloud, Oracle Cloud, etc.
- **Cloud platforms offer scalable infrastructure, managed DevOps tools, and automation capabilities that allow teams to:**
 - Deploy applications faster
 - Reduce manual effort
 - Handle dynamic workloads
 - Collaborate more efficiently

DevOps on the Cloud

Benefits of DevOps on the Cloud

- **Speed:** Faster software delivery with automation.
- **Scalability:** Easily handles growing user demands.
- **Reliability:** Automated testing, rollback, and monitoring.
- **Cost Efficiency:** Pay-as-you-use models reduce upfront infrastructure costs.
- **Global Reach:** Deploy applications to servers around the world in minutes.

Real-World Example: Netflix Using DevOps on AWS Cloud

Netflix is one of the best-known companies for using DevOps on the cloud, specifically AWS.

How Netflix Uses DevOps on AWS

Function	Tool/Service	Purpose
Infrastructure	AWS EC2, S3, Lambda	On-demand, scalable computing and storage
CI/CD	Jenkins + Spinnaker	Automate build, test, and deployment of microservices
Containerization	Titus (Netflix's custom orchestrator) + Docker	Runs containers at scale
Monitoring	Atlas (Netflix's tool) + AWS CloudWatch	Monitors real-time system metrics
Chaos Engineering	Chaos Monkey (part of Netflix's Simian Army)	Tests resilience by randomly terminating instances
IaC	Custom scripts and tools using AWS SDKs	Automates infrastructure creation

Result

Netflix can deploy hundreds of times per day with high reliability and minimal downtime across global cloud infrastructure.

More Real-World Examples

- **Spotify on Google Cloud**

- Uses GCP for data analytics and CI/CD with Jenkins and Kubernetes.
- Leverages Cloud Build and Stackdriver for monitoring.

- **Etsy on AWS**

- Migrated to AWS for better deployment pipelines and scaling.
- Uses Terraform + Jenkins + Kubernetes for seamless DevOps automation.

- **Adobe on Azure**

- Azure DevOps is used to manage code repos, pipelines, testing, and releases.
- Centralizes DevOps across distributed teams.